



Chapter 3 – Agile Software Development

Topics covered



- ✧ Overview of Agile
- ✧ Agile methods
- ✧ Agile development techniques

Agile Software Development



In 2001, Kent Beck and 16 other noted software developers, writers, and consultants [Bec01] (referred to as the “Agile Alliance”) signed the “Manifesto for Agile Software Development.” It stated:

We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:

Individuals and interactions over processes and tools

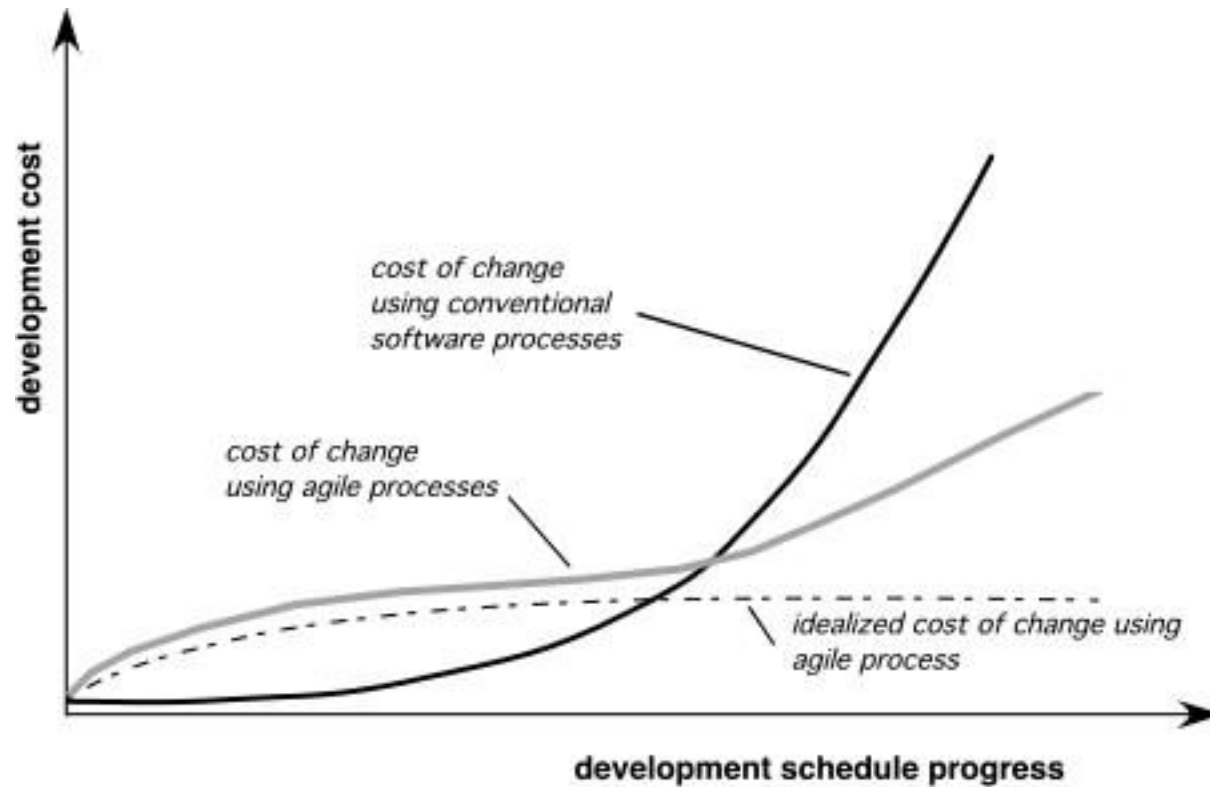
Working software over comprehensive documentation

Customer collaboration over contract negotiation

Responding to change over following a plan

That is, while there is value in the items on the right, we value the items on the left more.

Agility and the Cost of Change



Manifesto development!



✧ At the end of this class, will you **develop a manifesto, critically consider something else, or do both?**

Rapid software development



- ✧ **Plan-driven development** is essential for some types of system but does **not meet these business needs**.
 - **Rapid development** and delivery are now often the most important requirements for software systems
 - Businesses operate in a fast –changing requirement and it is **practically impossible to produce a set of stable software requirements**
 - **Software has to evolve quickly** to reflect changing business needs.
- ✧ **Agile development methods** emerged in the **late 1990s** whose aim was to **radically reduce the delivery time** for **working software systems**

Agile development - Common Characteristics

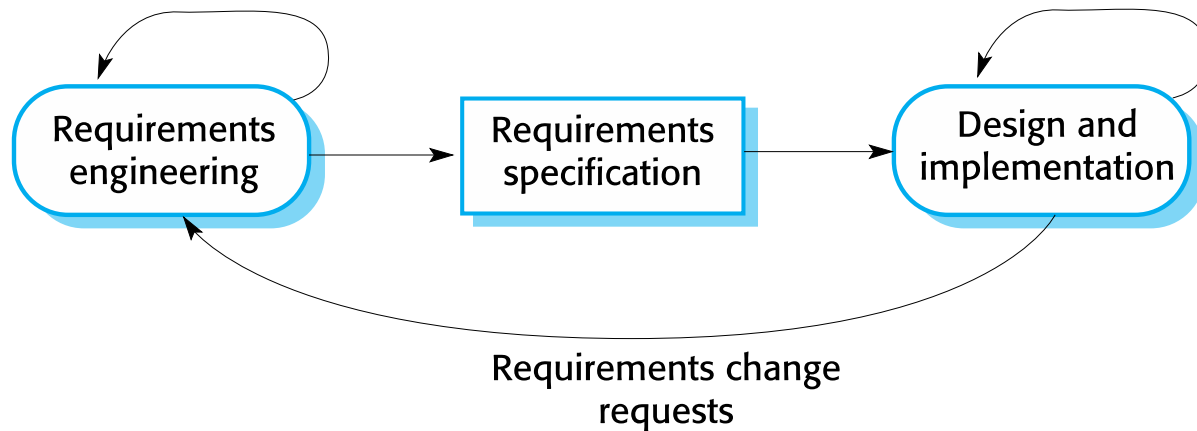


- ✧ Program specification, design and implementation are **inter-leaved**
- ✧ The system is developed as a **series of increments/versions**, with stakeholders involved in version specification and evaluation
- ✧ **Frequent delivery** of new versions for evaluation
- ✧ **Extensive tool support** (e.g. automated testing tools) used to support development.
- ✧ **Minimal documentation** – focus on working code

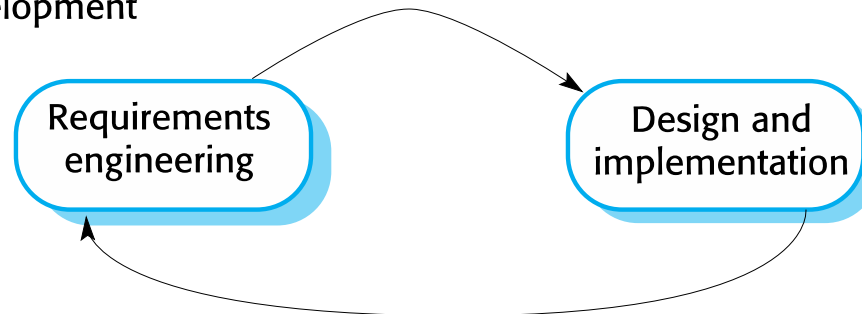
Plan-driven and agile development



Plan-based development



Agile development





Agile methods

Agile methods – Reducing Overheads



- ✧ Dissatisfaction with the **overheads** involved in software design methods of the 1980s and 1990s led to the creation of agile methods. These methods:
 - **Focus on the code** rather than the design
 - Are based on an **iterative approach** to software development
 - Are intended to **deliver working software quickly** and evolve it quickly to meet changing requirements.
- ✧ The **aim of agile methods is to reduce overheads** in the software process (e.g. **by limiting documentation**) and to be able to **respond quickly to changing requirements** without excessive rework.

Agile manifesto



- ✧ *We are uncovering better ways of developing software by doing it and helping others do it. Through this work, we have come to value:*
 - **Individuals and interactions** over processes and tools
 - Working software** over comprehensive documentation
 - Customer collaboration** over contract negotiation
 - Responding to change** over following a plan

- ✧ *That is, **while there is value in the items on the right, we value the items on the left more.***

Human Factors - Human Aspects of SE



- ✧ *The process molds to the needs of the people and team, not the other way around.*
- ✧ key traits must exist among the people on an agile team and the team itself:
 - **Competence.**
 - **Common focus.**
 - **Collaboration.**
 - **Decision-making ability.**
 - **Fuzzy problem-solving ability.**
 - **Mutual trust and respect.**
 - **Self-organization.**

What is “Agility”?



- ✧ Effective (rapid and adaptive) response to change
- ✧ Effective communication among all stakeholders
- ✧ Drawing the customer onto the team
- ✧ Organizing a team so that it is in control of the work performed
- ✧ Rapid, incremental delivery of software

Agility Principles



1. Our **highest priority is to satisfy the customer** through early and continuous delivery of valuable software.
2. **Welcome changing requirements**, even late in development. Agile processes harness change for the customer's competitive advantage.
3. **Deliver working software frequently**, from a couple of weeks to a couple of months, with a preference to the shorter timescale.
4. **Business people and developers must work together** daily throughout the project.
5. **Build projects around motivated individuals**. Give them the environment and support they need, and trust them to get the job done.
6. The most efficient and effective method of conveying information to and within a development team is **face-to-face conversation**.

Agility Principles



7. Working software is the primary measure of progress.
8. Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
9. Continuous attention to technical excellence and good design enhances agility.
10. Simplicity – the art of maximizing the amount of work not done – is essential.
11. The best architectures, requirements, and designs emerge from self-organizing teams.
12. At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.

Agile method applicability



- ✧ **Product development** where a software company is developing a **small or medium-sized product** for sale.
 - **Virtually all software products and apps are now developed using an agile approach**

- ✧ **Custom system development** within an organization, where there is a **clear commitment from the customer to become involved** in the development process and where there are **few external rules and regulations** that affect the software.



Class Exercise!

Class Exercise - A



- List five techniques/models of plan-driven methods and five techniques/models of agile methods.

Class Exercise - B



- ✧ Most projects include elements of plan-driven and agile processes. **Deciding on the balance depends on:**
- [01] Is it important to have a very **detailed specification and design before moving to implementation?** **If so, what method do you probably need to consider?**
 - [02] Is an **incremental delivery strategy**, where you deliver the software to customers and **get rapid feedback** from them, realistic? **What method do you probably need to consider?**
 - [03] **How large the system being developed is is a factor.** **Are Agile methods effective when a large, complex system requires formal communication?**

Class Exercise - B



- The type of system being developed is a factor.
 - [04] What method should be considered for systems that require a lot of analysis before implementation (e.g., a real-time system with complex timing requirements)?
- The expected system lifetime is a factor.
 - [05] What method should be considered for systems that require long-lifetime systems, and may require more design documentation to communicate the original intentions of the system developers to the support team?
- The available technology to support system development is a factor.
 - [06] What methods rely most on good tools to keep track of an evolving design
- The organization of the development team is a factor.
 - [07] What method should be considered if the development team is distributed or if part of the development is being outsourced, and you need to develop design documents to communicate across the development teams?

Class Exercise - B



- [08] What method should be considered if there are cultural or organizational issues that may affect the system development?
- How good the designers and programmers in the development team are is a factor
- [09] What method is most adequate to develop a system that requires higher-skilled programmers?
- A system subject to external regulation is a factor.
- [10] What method should be considered if a system has to be approved by an external regulator (e.g., the FAA approves software that is critical to the operation of an aircraft)?



Agile development techniques

Extreme programming



- ✧ A very influential agile method, developed in the late 1990s, that introduced a range of agile development techniques.
- ✧ Extreme Programming (XP) takes an 'extreme' approach to iterative development.
 - **New versions** may be built several times per day.
 - **Increments** are delivered to customers every 2 weeks.
 - All **tests** must be run for every build, and the build is only accepted if tests run successfully.

Extreme Programming (XP)



- ✧ The most widely used agile process, originally proposed by Kent Beck

The extreme programming release cycle

✧ [1] XP Planning

- Begins with the creation of “user stories” by listening
- Customer assigns a value to the story.
- Agile team assesses each story and assigns a cost
- Stories are grouped into a deliverable increment
- A commitment is made on the delivery date
- After the first increment, “project velocity” is used to help define subsequent delivery dates for other increments

Extreme Programming (XP)



✧ [2] XP Design

- Follows the **KIS principle**
- Encourage the use of **CRC cards** (see Chapter 8)
- For difficult design problems, suggests the creation of “**spike solutions**”—a design prototype
- Encourages “**refactoring**”—an iterative refinement of the internal program design

Extreme Programming (XP)



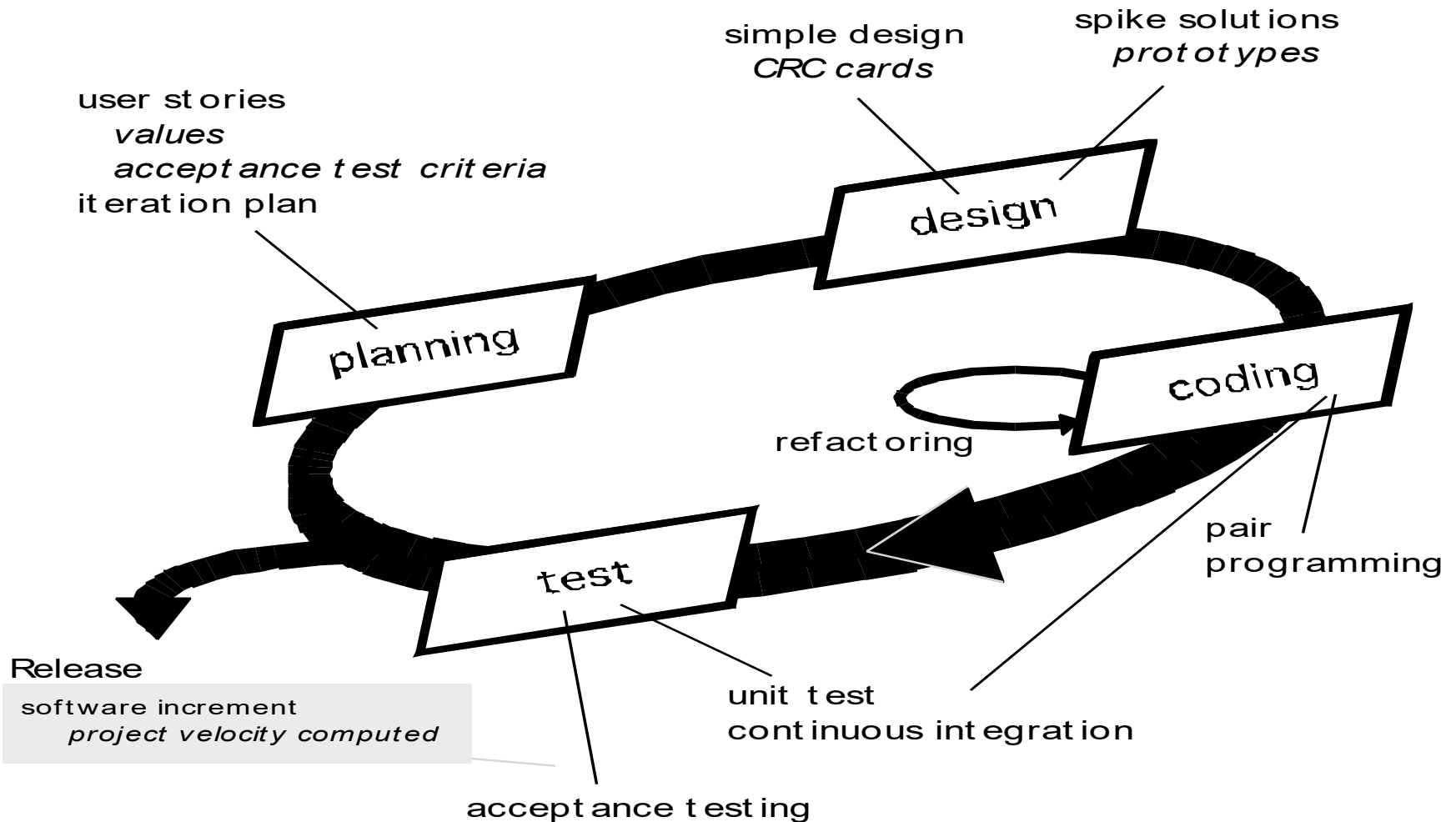
✧ [3] XP Coding

- Recommends the construction of a unit test for a story *before* coding commences
- Encourages “pair programming.”

✧ [4] XP Testing

- All unit tests are executed daily (whenever code is modified)
- “Acceptance tests” are defined by the customer and executed to assess customer-visible functionality

Extreme Programming (XP)





Class Exercise!

Exercise - C



✧ Scenario - a

- ✧ Imagine a software development team that has decided to implement **Pair Programming**, where two programmers work together on the same task. As the team **evaluates the effectiveness of this approach**, they notice that the productivity rate of these pairs is significantly higher than that of two programmers working independently. What could be **four potential reasons for this increased productivity when programmers collaborate in pairs?**

5th Exercise :



✧ Scenario - b

- ✧ A software development team at TechSolutions Inc. is transitioning from a plan-driven process to Agile methodology to enhance their responsiveness to client feedback and improve project delivery timelines. The team is currently working on a customer relationship management (CRM) system. Describe how the Extreme Programming (XP) process would use its framework activity to change the team's approach to developing the CRM system.

Scrum

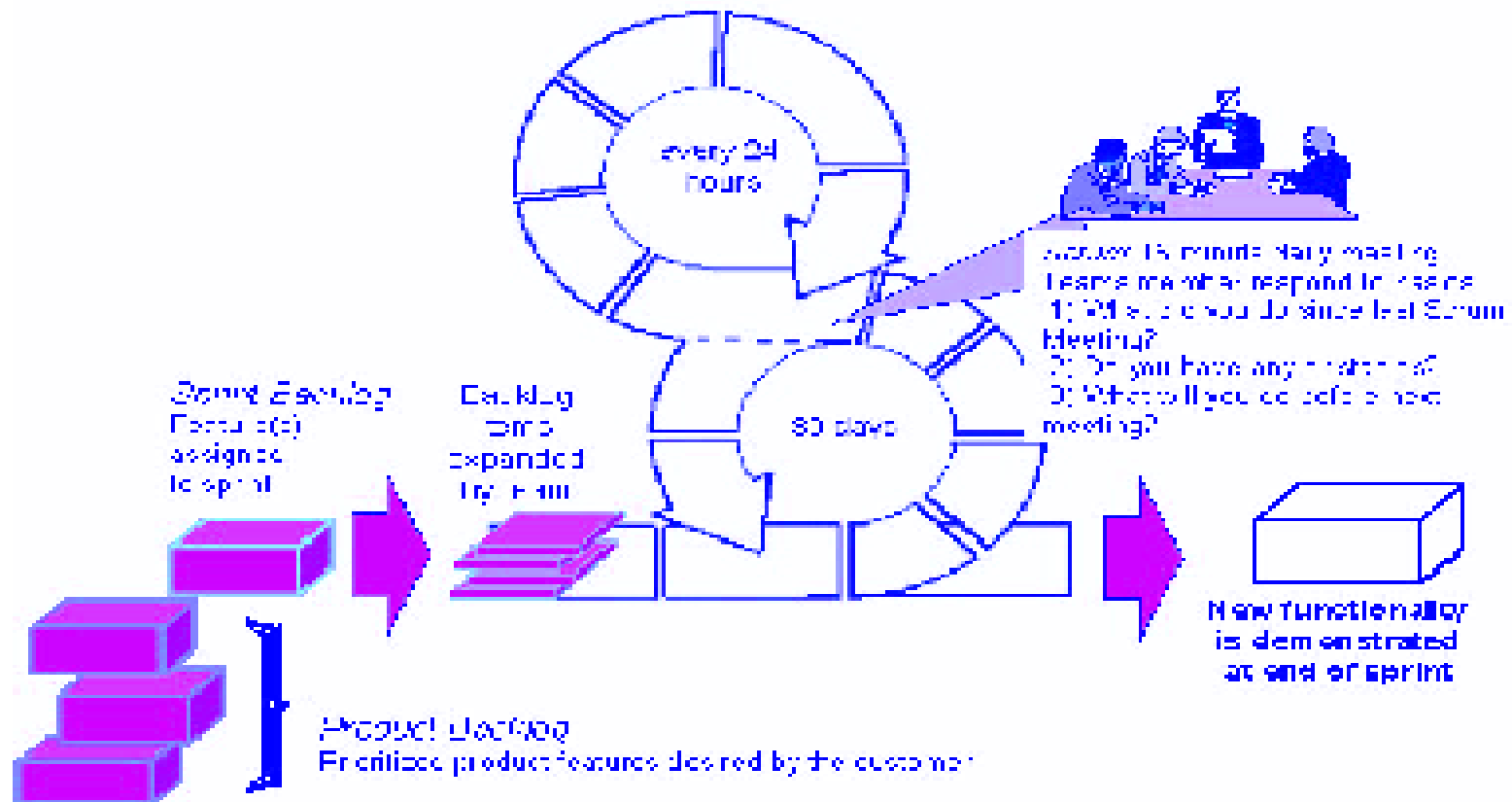


✧ Originally proposed by Schwaber and Beedle

✧ Scrum—distinguishing features

- Development work is partitioned into “**packets.**”
- **Testing and documentation are ongoing** as the product is constructed
- Work occurs in “**sprints**” and is derived from a “**backlog**” of existing requirements
- **Meetings are very short** and sometimes conducted without chairs
- “**Demos**” are delivered to the customer with the time-box allocated

Scrum



Scrum Process Flow (used with permission)

Scrum



- ✧ **Scrum** is an agile method that focuses on **managing iterative development** rather than specific agile practices.
- ✧ There are three phases in Scrum.
 - The initial phase is an **outline planning phase** where you establish the **general objectives** for the project and **design the software architecture**.
 - This is followed by a **series of sprint cycles**, where each cycle **develops an increment** of the system.
 - The **project closure phase** wraps up the project, completes required **documentation** such as system help frames and user manuals and assesses the lessons learned from the project.



Scrum terminology



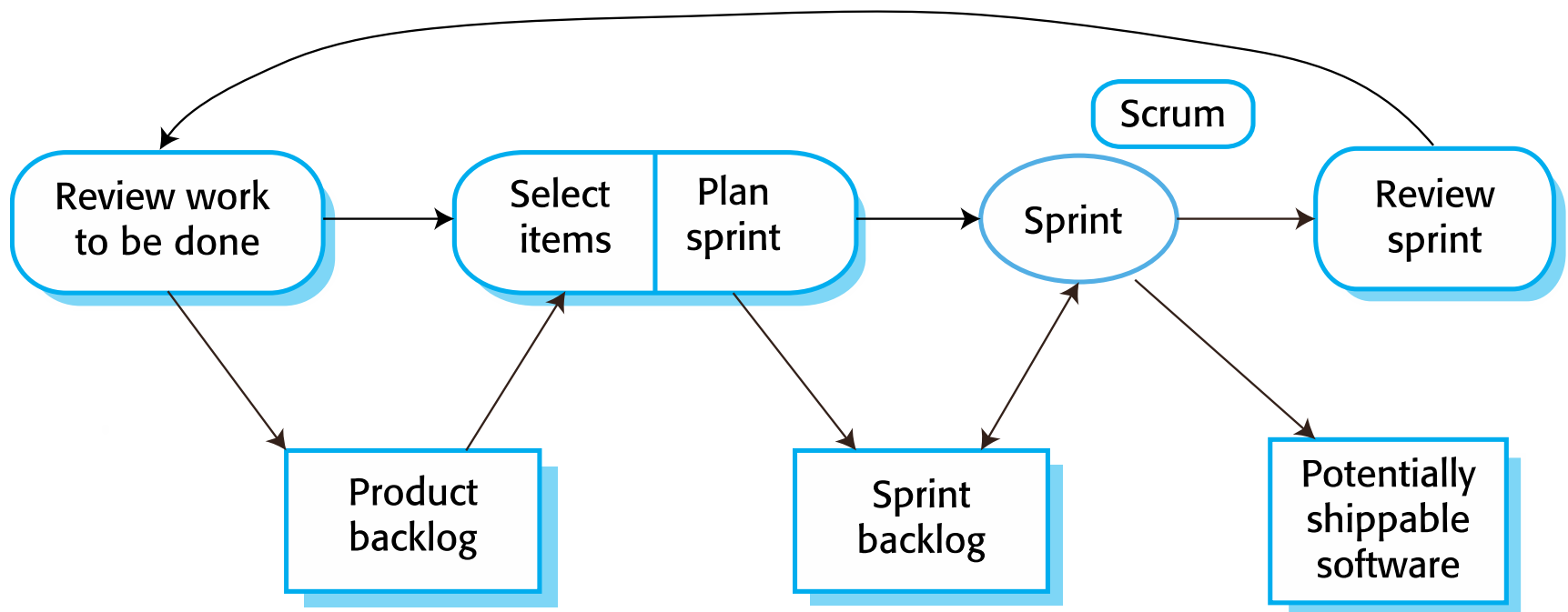
Scrum term	Definition
Development team	A self-organizing group of software developers , which should be no more than 7 people . They are responsible for developing the software and other essential project documents.
Potentially shippable product increment	The software increment that is delivered from a sprint. The idea is that this should be 'potentially shippable' which means that it is in a finished state and no further work , such as testing, is needed to incorporate it into the final product. In practice, this is not always achievable.
Product backlog	This is a list of 'to do' items which the Scrum team must tackle. They may be feature definitions for the software, software requirements, user stories or descriptions of supplementary tasks that are needed, such as architecture definition or user documentation.
Product owner	An individual (or possibly a small group) whose job is to identify product features or requirements, prioritize these for development and continuously review the product backlog to ensure that the project continues to meet critical business needs. The Product Owner can be a customer but might also be a product manager in a software company or other stakeholder representative.

Scrum terminology



Scrum term	Definition
Scrum	A daily meeting of the Scrum team that reviews progress and prioritizes work to be done that day . Ideally, this should be a short face-to-face meeting that includes the whole team.
ScrumMaster	The ScrumMaster is responsible for ensuring that the Scrum process is followed and guides the team in the effective use of Scrum . He or she is responsible for interfacing with the rest of the company and for ensuring that the Scrum team is not diverted by outside interference. The Scrum developers are adamant that the ScrumMaster should not be thought of as a project manager. Others, however, may not always find it easy to see the difference.
Sprint	A development iteration . Sprints are usually 2-4 weeks long .
Velocity	An estimate of how much product backlog effort that a team can cover in a single sprint . Understanding a team's velocity helps them estimate what can be covered in a sprint and provides a basis for measuring improving performance.

Scrum sprint cycle



The Scrum sprint cycle



- ✧ **Sprints** are fixed length, normally **2–4 weeks**.
- ✧ The **starting point** for planning is the **product backlog**, which is the list of work to be done on the project.
 - A requirements document
 - A list of user stories
 - Description of the software to be developed
- ✧ The **selection phase** involves all of the project team who work with the customer to select the **features** and **functionality** from the product backlog to be developed during the sprint.

The Sprint cycle



- ✧ Once these are agreed, the team **organize themselves to develop the software**.
- ✧ During this stage the team is isolated from the customer and the organization, with all communications channelled through the so-called '**Scrum master**'.
- ✧ The role of the **Scrum master** is to **protect the development team from external distractions**.
- ✧ At the end of the sprint, the **work done is reviewed and presented to stakeholders**. The next sprint cycle then begins.

Teamwork in Scrum



✧ The '**Scrum master**' is a facilitator:

- arranges daily meetings,
- tracks the backlog of work to be done,
- records decisions,
- measures progress against the backlog and
- communicates with customers and management outside of the team.

Teamwork in Scrum



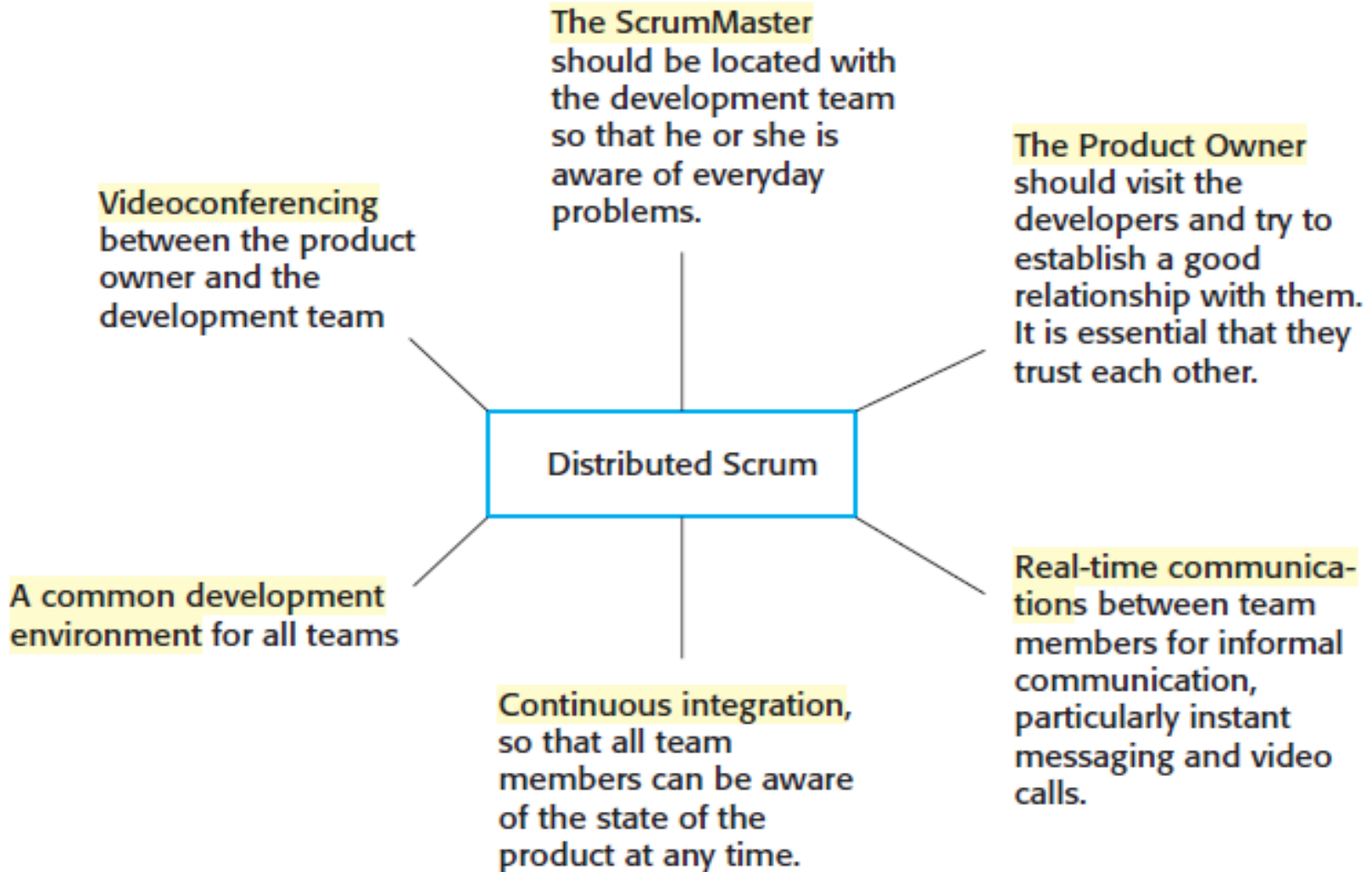
- ✧ The whole team attends **short daily meetings (Scrums)** where all team members share:
 - Information
 - Describe their progress since the last meeting
 - Raise problems that have arisen
 - Planned for the following day.
- Everyone on the team knows what is going on and, if problems arise, can re-plan short-term work to cope with them.

Scrum benefits



- ✧ The product is broken down into a set of **manageable** and **understandable chunks**.
- ✧ **Unstable requirements** do not hold up progress.
- ✧ The whole team have visibility of everything and consequently **team communication is improved**.
- ✧ Customers see **on-time delivery of increments** and gain **feedback on how the product works**.
- ✧ **Trust between customers and developers is established** and a positive culture is created in which everyone expects the project to succeed.

Distributed Scrum

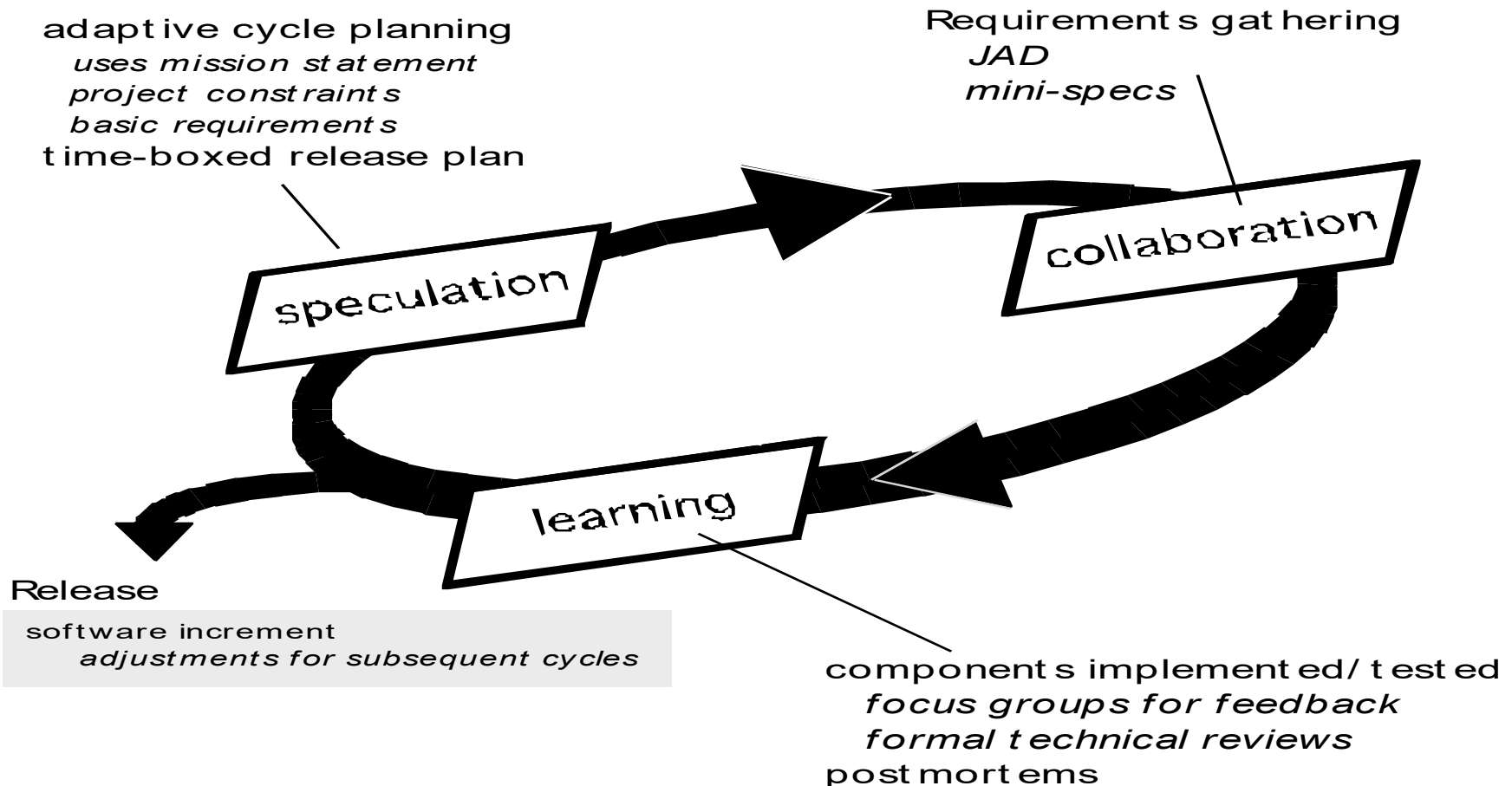


Adaptive Software Development



- ✧ Originally proposed by Jim Highsmith
- ✧ ASD — distinguishing features
 - Mission-driven planning
 - Component-based focus
 - Uses “time-boxing” (See Chapter 24)
 - Explicit consideration of risks
 - Emphasizes collaboration for requirements gathering
 - Emphasizes “learning” throughout the process

Adaptive Software Development

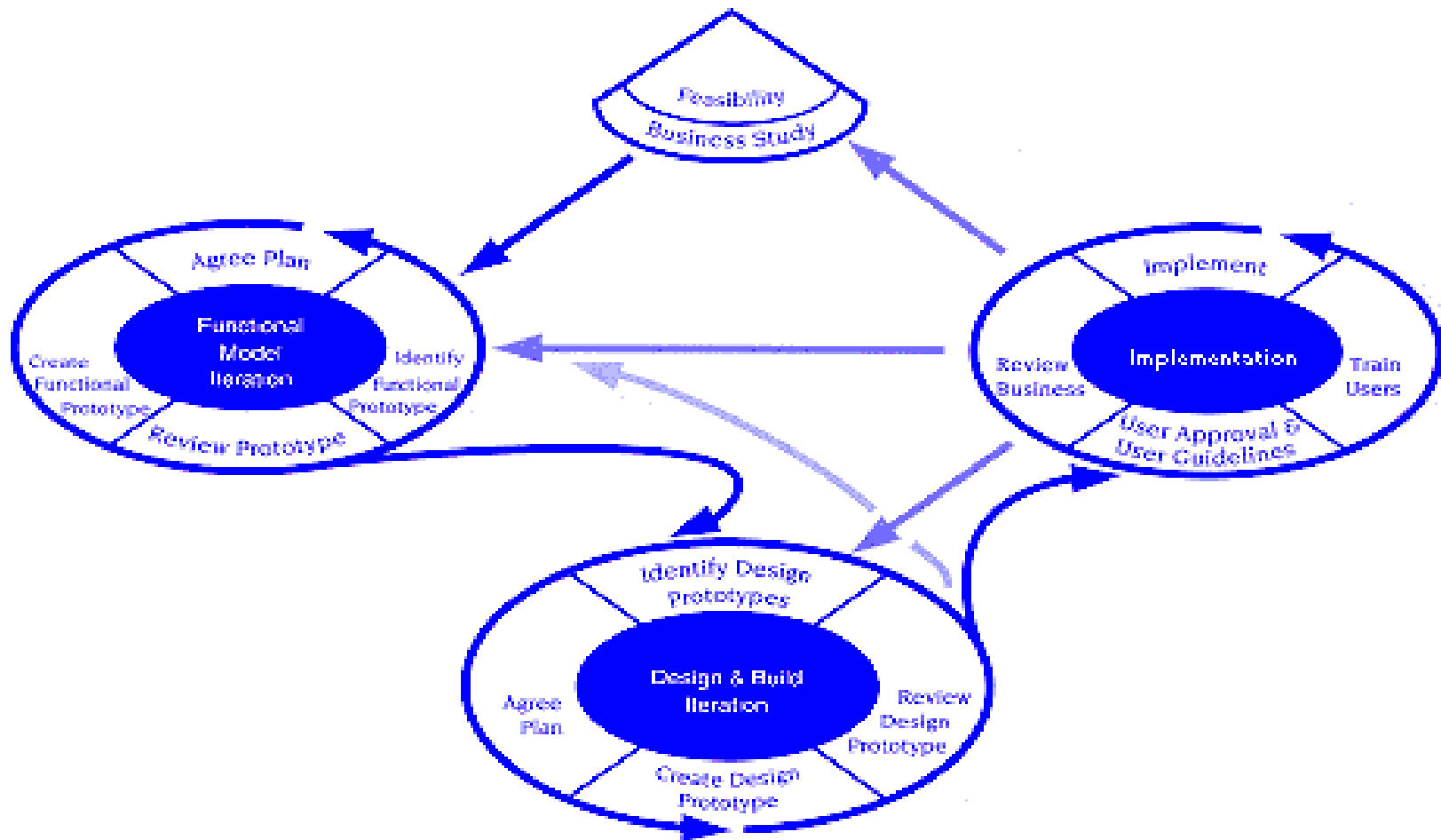


Dynamic Systems Development Method



- ✧ Promoted by the DSDM Consortium (www.dsdm.org)
- ✧ DSDM—distinguishing features
 - Similar in most respects to XP and/or ASD
 - Nine guiding principles
 - Active user involvement is imperative.
 - DSDM teams must be empowered to make decisions.
 - The focus is on frequent delivery of products.
 - Fitness for business purpose is the essential criterion for acceptance of deliverables.
 - Iterative and incremental development is necessary to converge on an accurate business solution.
 - All changes during development are reversible.
 - Requirements are baselined at a high level
 - Testing is integrated throughout the life-cycle.

Dynamic Systems Development Method



DSDM Life Cycle (with permission of the DSDM consortium)

Crystal



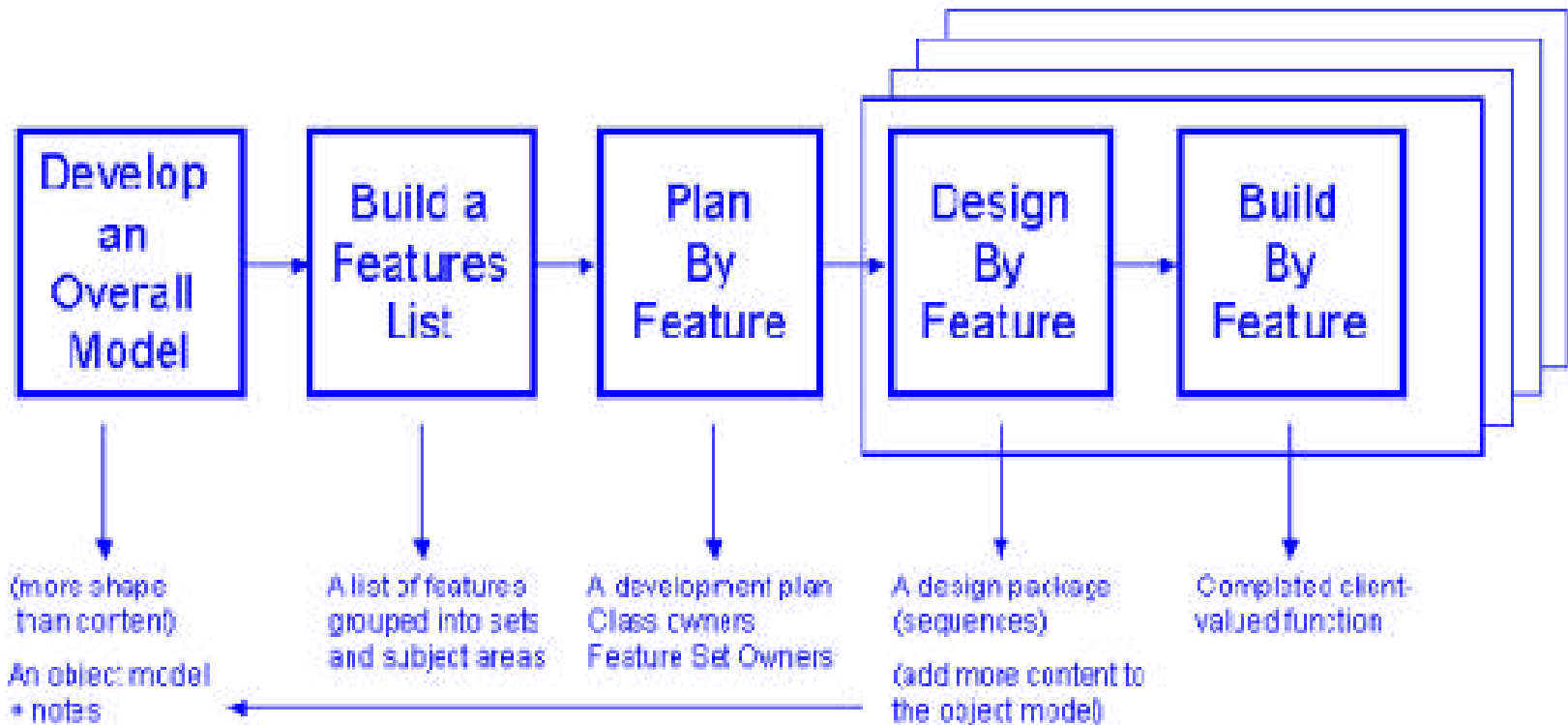
- ✧ Proposed by Cockburn and Highsmith
- ✧ Crystal—distinguishing features
 - Actually a **family of process models** that allow “**maneuverability**” based on problem characteristics
 - **Face-to-face communication** is emphasized
 - Suggests the use of “**reflection workshops**” to review the work habits of the team

Feature Driven Development



- ✧ Originally proposed by Peter Coad et al
- ✧ FDD—distinguishing features
 - Emphasis is on defining “features”
 - a *feature* “is a client-valued function that can be implemented in two weeks or less.”
 - Uses a *feature template*
 - <action> the <result> <by | for | of | to> a(n) <object>
 - A *features list* is created and “plan by feature” is conducted
 - Design and construction merge in FDD

Feature Driven Development



Reprinted with permission of Peter Coad

Agile Modeling



- ✧ Originally proposed by Scott Ambler
- ✧ Suggests a set of agile modeling principles
 - Model with a purpose
 - Use multiple models
 - Travel light
 - Content is more important than representation
 - Know the models and the tools you use to create them
 - Adapt locally